

# Can a clinic afford to run its own coding model?

## Dr. Findus, our startup in Berlin

German GPs must code every consultation.  
We read the note and propose the codes;  
the physician confirms each one. A legal  
requirement here. Reliability is the product:  
over-coding is fraud exposure, not error.

**Today that runs on hosted APIs.  
Could it run on our own hardware?**

An infrastructure question, not a model one.  
so we built the workload and measured it.

Qwen3 + LoRA on CodiEsp: 1 000 clinical case  
reports, ICD-10 labelled, CC-BY 4.0.

## 8 GB

of weights at 4B, before a single  
activation exists

## 2 489

tokens in the longest case.  
Sequence length is not free

## 3 backends

CPU · GPU · TPU, one JAX program,  
no per-backend reimplementation

## Where does the time actually go, and what does it take to run this at all?

*No patient data was used. Model accuracy earns no marks here. The object of study is the infrastructure.*

# One program. Three backends. All measured.

## Compilation

One JAX / Flax program.  
XLA lowers it to CPU, CUDA  
and TPU.  
LoRA via qwix, training via  
Tunix, no per-backend  
reimplementation.

## Orchestration

3 pinned Docker images.  
Kubernetes Jobs across two  
clusters.  
Kueue admission on the TPU  
pool. Every manifest states  
its CPU, memory and chips.

## Storage

gcsfuse CSI, implicit-dirs.  
An 8 GB checkpoint read  
from a bucket.  
The file cache was left as a  
knob rather than fixed -  
that knob became the  
experiment.

| Backend | Hardware               | Host arch   | How the code got there     |
|---------|------------------------|-------------|----------------------------|
| CPU     | 32-core x86_64, 31 GiB | x86_64      | bare node                  |
| GPU     | NVIDIA GH200 480 GB    | ARM64 Grace | public image + ConfigMap   |
| TPU     | v5e 2x4, 8 chips       | x86_64      | private registry + gcsfuse |

*Note the middle row: the GH200's host is ARM64, not x86. That single fact cost us five separate blockers.*

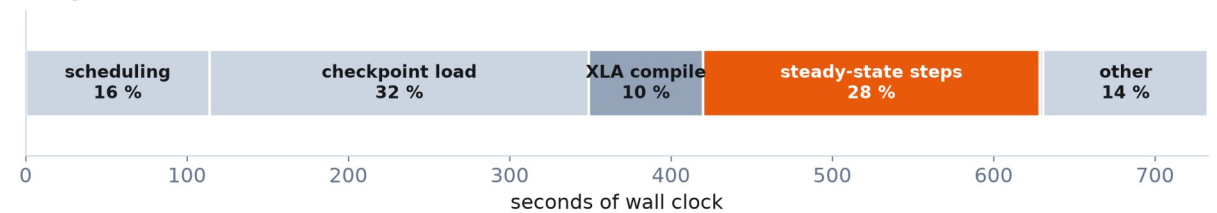
# We did not time the job. We took it apart.

Every run takes its own wall clock apart into six phases: scheduling, image pull, checkpoint load, XLA compile, the steady-state steps, checkpoint write.

Batch and sequence sweeps walked until the job dies. A failed cell is recorded, never dropped. The boundary is the measurement.

One schema, one JSON per run. Every figure here reads those files and nothing else.

Only 28 % of 733 s is arithmetic



## The schema caught a lie

One run reported 93  $\mu$ s per step and 11 M tokens/s. Impossible, yet it carried status "ok". Its own notes flagged that it may have timed dispatch, not completion. Discarded, repeated.

# One Hopper chip kept up with eight TPU chips.

| Qwen3-0.6B · bs 1 · seq 1024 | CPU     | GPU GH200 | TPU v5e 2x4 |
|------------------------------|---------|-----------|-------------|
| chips                        | -       | 1         | 8           |
| median step                  | 18.99 s | 0.0798 s  | 0.0800 s    |
| throughput                   | 54      | 12,834    | 12,804      |
| speedup vs CPU               | 1.0x    | 238x      | 237x        |
| peak memory                  | 66 %    | 5.0 %     | 7.3 %       |
| USD / 1 000 steps            | \$6.05  | \$0.131   | \$1.03      |

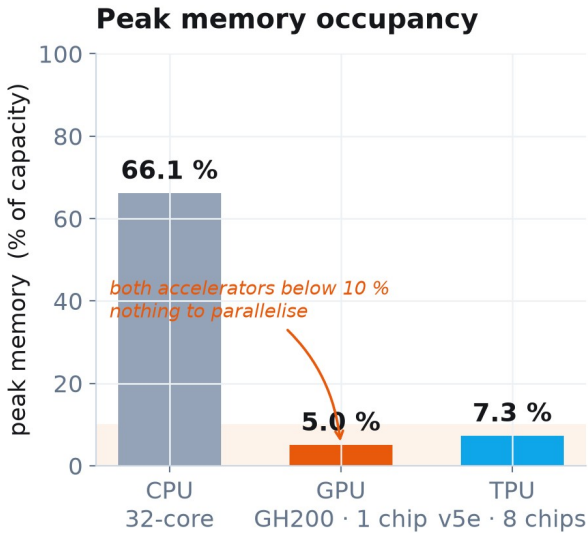
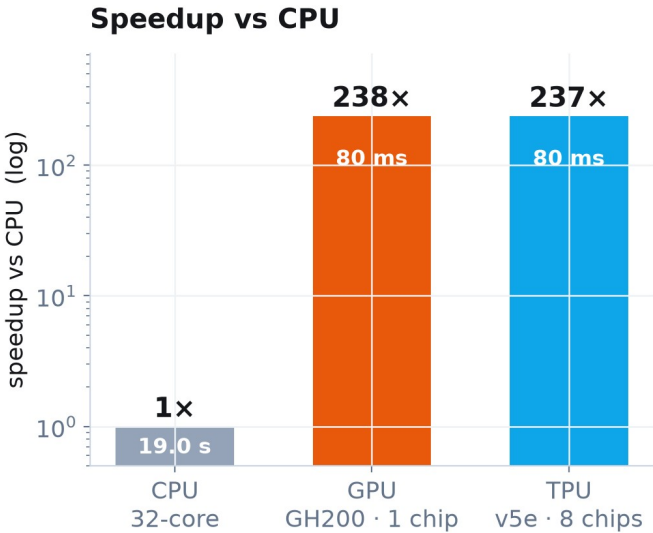
0.23 %

difference in step time between one GH200 and eight v5e chips

14.6x

faster checkpoint load from one manifest line, 31 % off wall clock

One Hopper chip keeps pace with eight TPU chips, because neither is busy



The control

Median step time before and after the cache:

1.75122 s to 1.75117 s

The computation did not move. Only I/O did, so the saving is attributable, not noise.

# The chip was never the bottleneck.

## 28 %

**of wall clock is arithmetic**

Of 733 s, only 209 s computes.  
The biggest single phase is  
reading the checkpoint.

## 6.7×

**more parameters, CPU stopped**

0.6B to 4B did not make the CPU  
6.7× slower. XLA never finished  
compiling; the kernel killed it  
at 31.5 GB.

## 1 of 36

**dependencies were TPU-bound**

The GH200 sat idle behind an  
ARM64 host, a QEMU segfault  
and a registry 403. Swapping  
one pin is the whole port.

## Scaling recommendation

Do not scale out. Amortise: raise batch only until the chip is occupied, then attack fixed cost: persistent compile caches, warm workers instead of a pod per job, file cache on anything reading a checkpoint.

## What this means for Dr. Findus

At \$0.131 per 1 000 steps on one GH200,  
8× cheaper than eight TPU chips at the same  
speed, self-hosting is not the cost problem we  
assumed. The real cost is fixed overhead per job:  
an architecture decision, not a hardware purchase.